



ARCHITECTURE DOCUMENT

Document 06 of 09 · B2B Data Engineering Platform



Describes the solution architecture: goals, constraints, context, building blocks, runtime behaviour, deployment, key decisions and quality requirements.

Document ID	B2B-DEP-06
Version	1.4
Status	Final
Date	05/10/2026
Prepared by	Syed Kazim Raza
Reviewed by	[Team Lead]

05/10/2026

DATAROPES.AI

Data Engineering Internship · B2B Data Engineering Platform

Document Control

Revision History

Version	Date	Author	Description of Change
0.1	-	Syed Kazim Raza	Template created
1.0	29/09/2026	Syed Kazim Raza	Architecture of the completed platform
1.1	30/09/2026	Syed Kazim Raza	Updated to the rebuilt data; staging emptied after each run; quarantine table; test results
1.2	04/10/2026	Syed Kazim Raza	Architecture, data-flow and end-to-end diagrams redrawn; Power BI shown in Import mode; metadata and quarantine shown
1.3	05/10/2026	Syed Kazim Raza	Architecture and end-to-end diagrams redrawn in container style (technology logos, numbered run order, results per stage)
1.4	05/10/2026	Syed Kazim Raza	Formatting to documentation standard: table column widths, numbered figure and table captions with lists, figure descriptions, complete acronym list

Approvals

Role	Name	Signature	Date
Prepared by	Syed Kazim Raza		
Reviewed by	[Team Lead]		
Approved by	[Manager]		

Related Documents

Ref	Document	Location
R1	Business Requirements Document (BRD)	Provided by DataRopes
R2	Project Document (technical reference)	docs/deliverables/project_document.md
R3	Project Plan	docs/deliverables/project_plan.md
R4	README / Deployment Guide	README.md

Contents

Document Control 2

Contents 3

List of Figures..... 4

List of Tables..... 5

1. Introduction 6

 1.1 Purpose 6

 1.2 Scope 6

 1.3 Intended Audience 6

 1.4 Definitions & Acronyms 6

2. Goals & Quality Objectives 7

 2.1 Requirements Overview 7

 2.2 Quality Goals 7

 2.3 Stakeholders 7

3. Constraints..... 8

4. System Context & Scope..... 9

5. Solution Strategy 10

6. Building Block View 11

7. Runtime View 13

 7.1 Scenario: Initial Load (2-year backfill) 15

 7.2 Scenario: Daily Incremental Run 15

 7.3 Scenario: Failure and Re-run 15

8. Deployment View 16

9. Cross-cutting Concepts..... 16

 9.1 Change Data Capture & Watermarks 16

 9.2 Idempotency & Transactions..... 17

 9.3 Slowly Changing Dimensions 17

 9.4 Data Quality 17

 9.5 Logging, Monitoring & Alerting 17

 9.6 Security & Configuration 18

10. Architecture Decisions..... 18

11. Quality Requirements (Non-Functional) 18

12. Risks & Technical Debt..... 19

13. Glossary 19

List of Figures

Figure 1: End-to-end flow — from seeded data to dashboard 9

Figure 2: System architecture..... 11

Figure 3: Table-level data flow 14

Figure 4: Staging tables are empty after a successful run..... 16

Figure 5: Rejected rows by table and reason 17

List of Tables

Table 1: Intended Audience..... 6

Table 2: Definitions & Acronyms 6

Table 3: Quality Goals..... 7

Table 4: Stakeholders 7

Table 5: Constraints..... 8

Table 6: System Context & Scope 10

Table 7: Solution Strategy 10

Table 8: Building Block View..... 12

Table 9: Deployment View 16

Table 10: Architecture Decisions..... 18

Table 11: Quality Requirements (Non-Functional)..... 18

Table 12: Risks & Technical Debt..... 19

Table 13: Glossary 19

1. Introduction

1.1 Purpose

This document describes the architecture of the B2B Data Engineering Platform (structure based on arc42).

1.2 Scope

In scope:

- End-to-end architecture from data generation to Power BI
- Components, data flow, deployment, cross-cutting concepts, decisions

Out of scope:

- Column-level mappings (SMX)
- Detailed code (Development Document)

1.3 Intended Audience

Table 1: Intended Audience

Audience	How they use this document
Team Lead / Architects	Review design and decisions
Data Engineers	Understand how components fit
New joiners	Onboard quickly

1.4 Definitions & Acronyms

Table 2: Definitions & Acronyms

Term	Definition
ADR	Architecture Decision Record: a recorded design decision with alternatives and rationale (ADR-01 ...).
B2B	Business-to-Business: companies selling to other companies.
BI	Business Intelligence: reporting and dashboards, here Power BI.
BRD	Business Requirements Document: here the Project Requirements Document provided by DataRopes.
CDC	Change Data Capture: only new or changed rows are extracted on each run.
CI/CD	Continuous Integration / Continuous Delivery: automated build, test and deployment (out of scope for this project).
CSV	Comma-Separated Values: the text-file format of the marketing leads and web logs feeds.
DDL	Data Definition Language: SQL that creates schemas, tables, indexes and views.
DEP	Data Engineering Platform: prefix of the document IDs (B2B-DEP-01 ...).
ELT	Extract-Load-Transform: raw data is loaded first, then transformed inside the database with SQL.
ETL	Extract, Transform, Load: data is transformed before it is loaded (the alternative to ELT).
FK	Foreign Key: a column that references the primary key of another table.
HTTP	Hypertext Transfer Protocol: the web protocol recorded in the web logs (GET, POST ...).
ID	Identifier.
Idempotent	Running the same load twice gives the same result (no duplicates).
IF	Interface: IF-01 ... IF-03 number the interfaces in the Interface Specification.
KPI	Key Performance Indicator: a business measure shown on the dashboard (KPI-01 ... KPI-15).
MB	Megabyte.
NFR	Non-Functional Requirement: a quality requirement such as performance or reliability.
ODBC	Open Database Connectivity: the driver standard Python uses to connect to SQL Server.
PK	Primary Key: the column(s) that uniquely identify a row.

Term	Definition
SCD Type 2	Keeps a new dimension row for every change so history is preserved.
SMX	Source-to-Target Mapping workbook (deliverable 05, Excel).
SQL	Structured Query Language.
Watermark	The last extracted updated_at per table, used as the starting point of the next extract.

2. Goals & Quality Objectives

2.1 Requirements Overview

The Project Requirements Document asks for a local B2B data platform on PostgreSQL that:

- ingests **3 sources**: an SQL Server operational database, web server logs and marketing leads;
- stages raw data in PostgreSQL with **CDC** (only changed rows, tracked by updated_at watermarks);
- builds a **star constellation-schema warehouse** with an SCD Type 2 company dimension and date-partitioned facts;
- answers **15 KPIs** through SQL views, shown in Power BI;
- runs end-to-end through **local orchestration** with monitoring and notifications, idempotent loads and no CI/CD.

The data is synthetic: Python generators create 2 years of history and then one new day of activity per incremental run.

2.2 Quality Goals

Table 3: Quality Goals

Priority	Quality Goal	Scenario / Measure
1	Correctness & idempotency	Re-running any stage gives identical results; all warehouse loads use MERGE.
2	Traceability	Every rejected row is kept with a reason (rejected_records) and every run is logged in metadata.pipeline_run_log.
3	Efficiency	Only changed rows are extracted (CDC); facts are partitioned by month. An incremental run takes about 3 minutes.
4	Simplicity	One SQL script per table and step; runs on a single laptop with free tools.

2.3 Stakeholders

Table 4: Stakeholders

Role	Name	Expectations
Team Lead (DataRopes)	Reviewer	A working, documented pipeline that meets the requirements
Data Engineering Intern	Syed Kazim Raza	Design, build and document the platform
Business users	Dashboard consumers	Reliable KPIs in Power BI, refreshed after each run

3. Constraints

Table 5: Constraints

Constraint	Type	Explanation
Local platform	Technical	Everything runs on one Windows machine: SQL Server Express, PostgreSQL, Python, Power BI Desktop.
PostgreSQL 15+	Technical	Required for the MERGE statement used by all loads (installed: 18.4).
Synthetic data	Organisational	No production data is available; Python generators create realistic data.
No CI/CD	Organisational	Scripts are executed manually through the orchestrator (requirements document).
No automated test framework	Organisational	Validation is done with SQL test queries after end-to-end runs (requirements document).

4. System Context & Scope

Figure 1 shows the end-to-end flow of one pipeline run, from the data generators to the dashboard, with the results of the final run.

End-to-End: From Seeded Data to Dashboard

What happens to the data on one run of run_pipeline.py, with the numbers from the final run (30/09/2026)

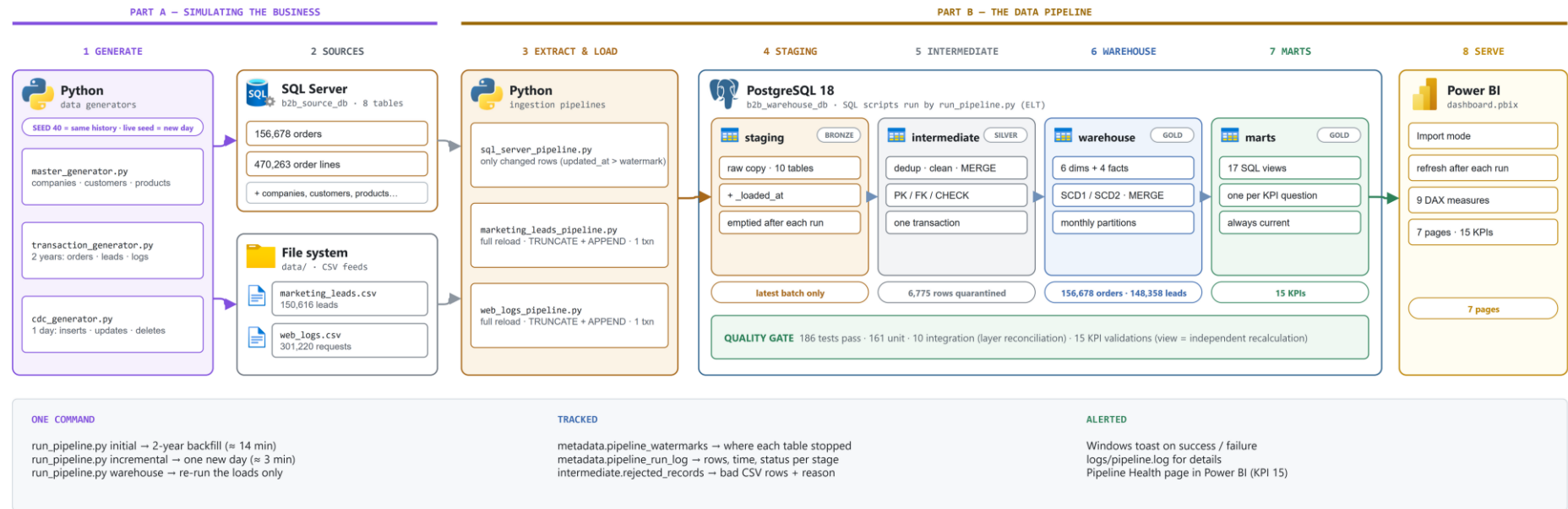


Figure 1: End-to-end flow — from seeded data to dashboard

Table 6: System Context & Scope

External System / Actor	Interface	Data Exchanged
SQL Server b2b_source_db	IF-01: SQL query (pyodbc), incremental	Companies, customers, suppliers, products, orders, order items
marketing_leads.csv	IF-02: file read, full snapshot	Marketing leads, campaigns, funnel stage
web_logs.csv	IF-03: file read, full snapshot	HTTP requests: device, browser, status, bytes
Power BI Desktop	PostgreSQL connector on schema marts	15 KPI views

5. Solution Strategy

Table 7: Solution Strategy

Decision Area	Approach	Rationale
Integration style	ELT: load raw data into PostgreSQL first, transform with SQL	Raw data lands before it is cleaned; SQL is the best tool for set-based transforms
Layering	staging -> intermediate -> warehouse -> marts	Each layer has one job: land, clean, model, serve
Modelling	Kimball galaxy schema: 4 facts sharing conformed dimensions	Simple star joins for Power BI; several business processes in one model
Incremental loads	Watermark CDC on updated_at + idempotent MERGE	Required by the requirements document; safe to re-run
Orchestration	Python run_pipeline.py (initial / incremental) with run log and Windows notifications	Local, no CI/CD; one command runs everything
Serving	SQL views in schema marts, consumed by Power BI	Business logic stays in SQL and is reusable

6. Building Block View

Figure 2 shows the building blocks: each technology with the components it runs, and the numbered order of a pipeline run.

B2B E-Commerce Data Platform — System Architecture

Python generates and ingests the data · PostgreSQL transforms it with SQL (ELT) · Power BI imports the marts · everything runs on one Windows 11 workstation

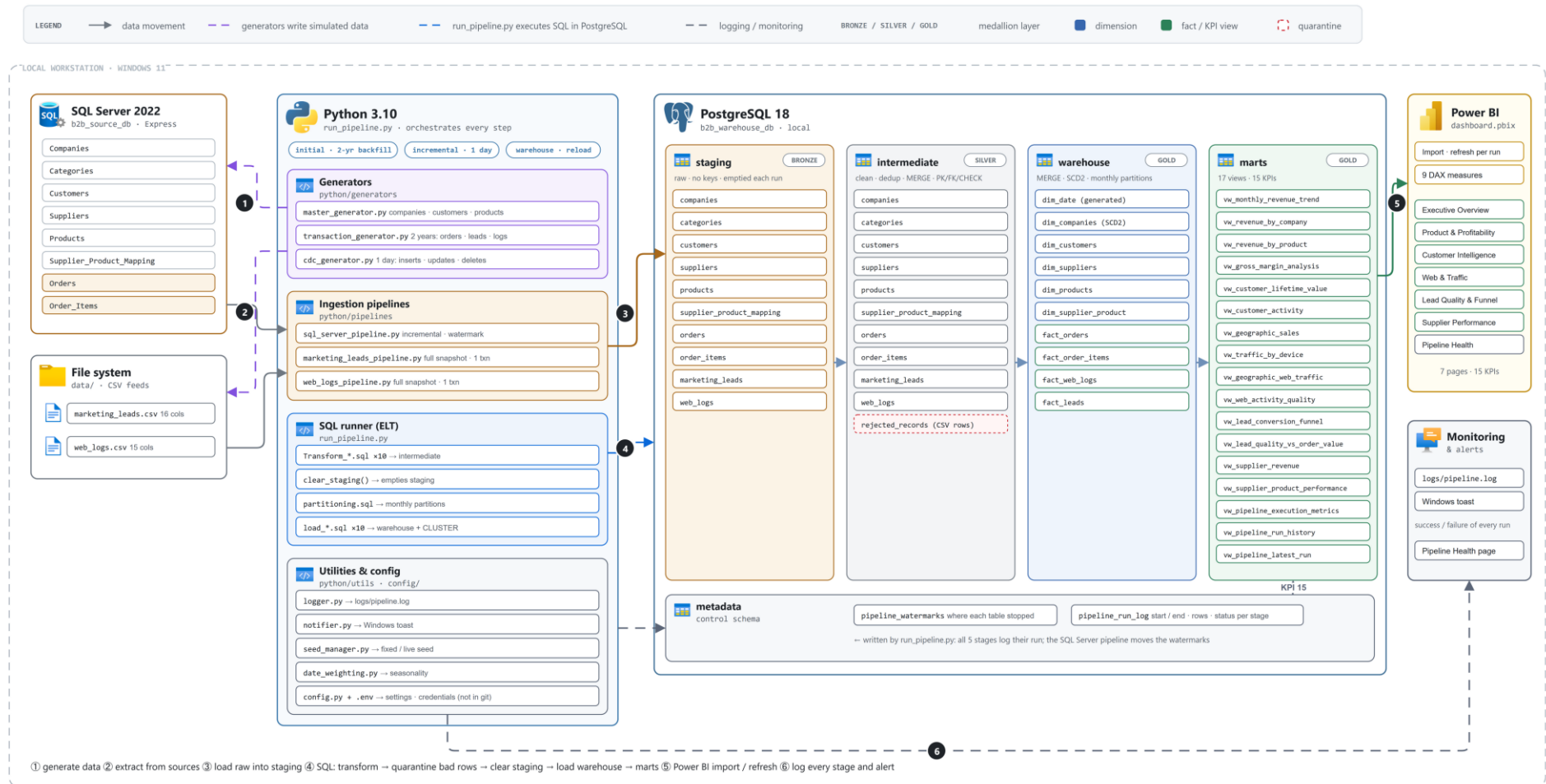


Figure 2: System architecture

Table 8: Building Block View

Component / Layer	Responsibility	Technology	Artifacts
Data generators	Create 2 years of history, then one new day per run (new rows + updates)	Python, Faker	python/generators/
Source layer	Operational database and two CSV feeds	SQL Server 2022 Express, CSV files	sql_server/, data/
Ingestion pipelines	Extract changed rows / files and append to staging	Python, pandas, SQLAlchemy	python/pipelines/*_pipeline.py
Staging	Raw copy of the latest batch + _loaded_at; emptied after each run	PostgreSQL	postgresql/staging/
Intermediate	Deduplicate, clean, validate; latest version per key; bad CSV rows to rejected_records	PostgreSQL SQL (MERGE)	postgresql/intermediate/transform/
Warehouse	6 dimensions (SCD1/SCD2) and 4 partitioned facts	PostgreSQL SQL (MERGE)	postgresql/warehouse/
Marts & reporting	15 KPI views and the Power BI dashboard	PostgreSQL views, Power BI	postgresql/marts/, dashboard.pbix
Orchestration & monitoring	Run all stages in order, log runs, notify	Python	run_pipeline.py, python/utils/, metadata schema

7. Runtime View

Figure 3 traces every table from staging through intermediate and warehouse to the mart views.

B2B E-Commerce Data Platform — Data Flow

Table-level lineage from raw sources to BI-ready Marts views

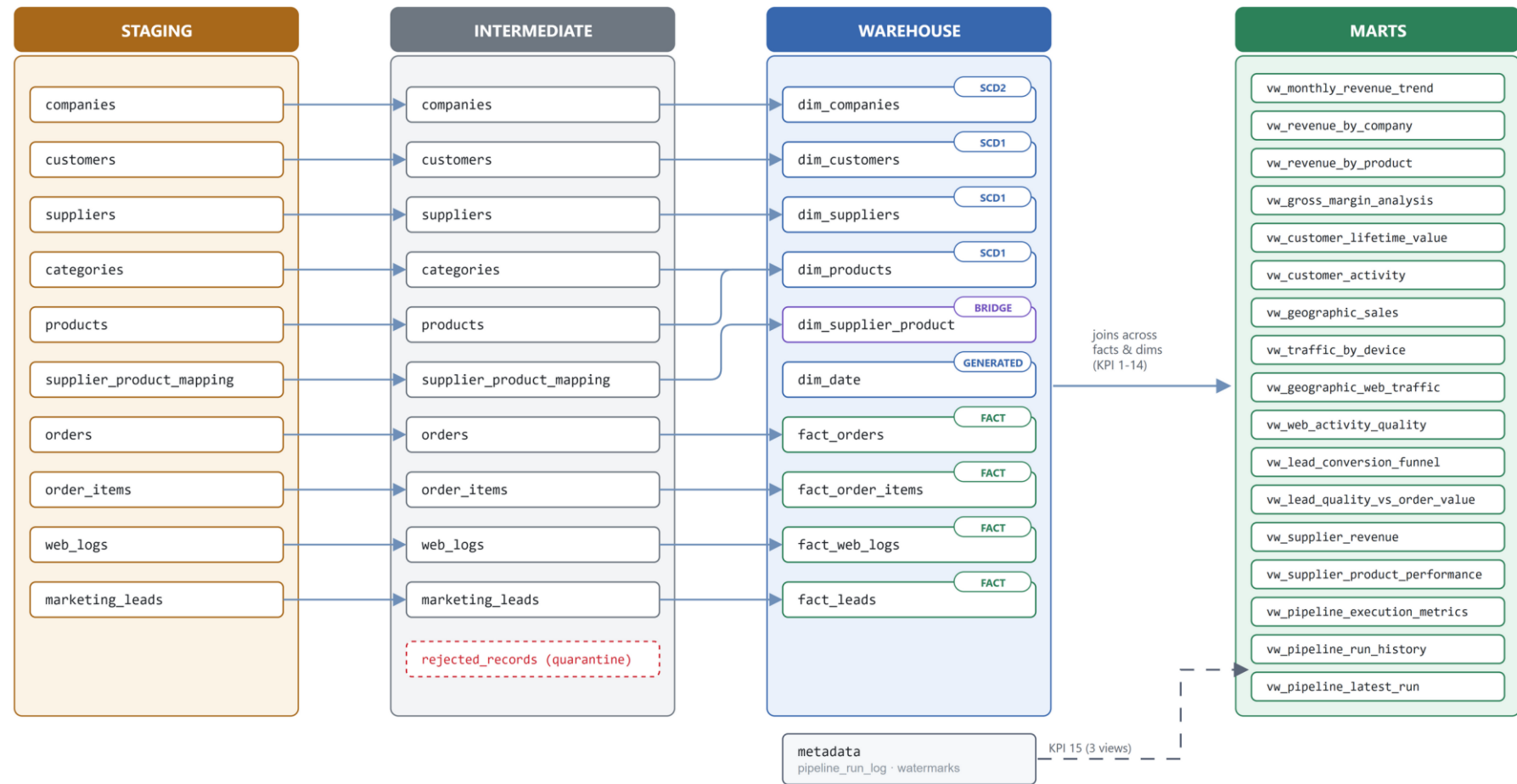


Figure 3: Table-level data flow

7.1 Scenario: Initial Load (2-year backfill)

1. Run `python -m python.pipelines.run_pipeline initial`.
2. Generators create master data (companies, customers, products ...) and 2 years of orders, leads and web logs; SQL Server is loaded and the two CSV files are written.
3. The three ingestion pipelines load everything into staging (no watermark yet = full extract) and store the first watermarks.
4. The 10 intermediate scripts clean and validate the data in one transaction; invalid CSV rows go to `intermediate.rejected_records`.
5. Staging is checked and emptied; the warehouse load fills `dim_date`, the dimensions, creates the monthly partitions and loads the four facts.
6. A Windows notification reports success; Power BI can be refreshed.

7.2 Scenario: Daily Incremental Run

1. Run `python -m python.pipelines.run_pipeline incremental`.
2. The CDC generator simulates one day: 180-230 new orders, 180-230 leads, 370-450 web logs (dated today) and updates to companies, customers, orders, leads and products.
3. The SQL Server pipeline extracts only rows with `updated_at` after the watermark; the CSV pipelines reload their files.
4. Intermediate MERGE inserts new keys and updates changed ones; staging is emptied; the warehouse applies SCD1/SCD2 and merges the facts.
5. Run log and notification are written. Typical duration: about 3 minutes.

7.3 Scenario: Failure and Re-run

If any stage fails, the orchestrator stops, the run is logged in `metadata.pipeline_run_log` with status 'failed' and the error message, and a failure notification appears. Watermarks are only moved after a successful load, and the intermediate stage runs in one transaction that is rolled back on error; staging is only emptied after it succeeded. Running the same command again therefore picks up the same data and, because every load is a MERGE, produces the correct result without duplicates.

Figure 4 is the evidence that staging is emptied after a successful run: every staging table returns 0 rows.

The screenshot shows a SQL query in a dark-themed editor. The query is a UNION ALL of five SELECT statements, each counting rows in a staging table. Below the query, the 'Data Output' tab is active, showing a table with two columns: 'staging_table' (text) and 'rows_left' (bigint). The table contains five rows, all with a value of 0 in the 'rows_left' column, indicating that the staging tables are empty.

```

13 SELECT 'orders' AS staging_table, COUNT(*) AS rows_left FROM staging.orders
14 UNION ALL
15 SELECT 'order_items', COUNT(*) FROM staging.order_items
16 UNION ALL
17 SELECT 'marketing_leads', COUNT(*) FROM staging.marketing_leads
18 UNION ALL
19 SELECT 'web_logs', COUNT(*) FROM staging.web_logs
20 UNION ALL
21 SELECT 'companies', COUNT(*) FROM staging.companies;
22

```

	staging_table text	rows_left bigint
1	orders	0
2	order_items	0
3	marketing_leads	0
4	web_logs	0
5	companies	0

Figure 4: Staging tables are empty after a successful run

8. Deployment View

Table 9: Deployment View

Node / Environment	Software	Purpose
Developer laptop (Windows 11)	SQL Server 2022 Express + ODBC Driver 17	Source database b2b_source_db
Developer laptop (Windows 11)	PostgreSQL 18.4	b2b_warehouse_db: staging, intermediate, warehouse, marts, metadata
Developer laptop (Windows 11)	Python 3.10 + requirements.txt	Generators, pipelines, orchestrator
Developer laptop (Windows 11)	Power BI Desktop	Dashboard (dashboard.pbix) on schema marts

9. Cross-cutting Concepts

9.1 Change Data Capture & Watermarks

For SQL Server, each table has a watermark (last loaded updated_at) in metadata.pipeline_watermarks. The extract reads only rows with updated_at later than the watermark, and the watermark moves forward only after a successful load. The CSV files have no reliable change column, so they are reloaded completely on each run. Implemented in sql_server_pipeline.py and pipeline_utils.py.

9.2 Idempotency & Transactions

Every intermediate and warehouse load is a MERGE on the business key: new rows are inserted, changed rows updated, unchanged rows left alone. The intermediate stage runs all 10 scripts in one transaction. Re-running a stage on the same data therefore never creates duplicates.

9.3 Slowly Changing Dimensions

dim_companies is SCD Type 2 (as required): a change to name, type, tax id, rating or location closes the current row and adds a new one, so orders are always reported with the company data valid at order time (2,505 versions for 2,500 companies). The other dimensions are SCD Type 1 (overwrite).

9.4 Data Quality

Staging accepts everything. The intermediate layer keeps only the latest version of each record, trims text, and checks mandatory fields, allowed values, ranges and parent records. Invalid CSV rows are saved in intermediate.rejected_records with a reason (1.5% are broken on purpose to test this). Database constraints in intermediate and warehouse (PK, FK, CHECK) act as a second safety net, and 161 unit tests, 10 integration tests and 15 KPI checks validate each layer.

Figure 5 shows the rejected rows per table and reason, as stored in intermediate.rejected_records.

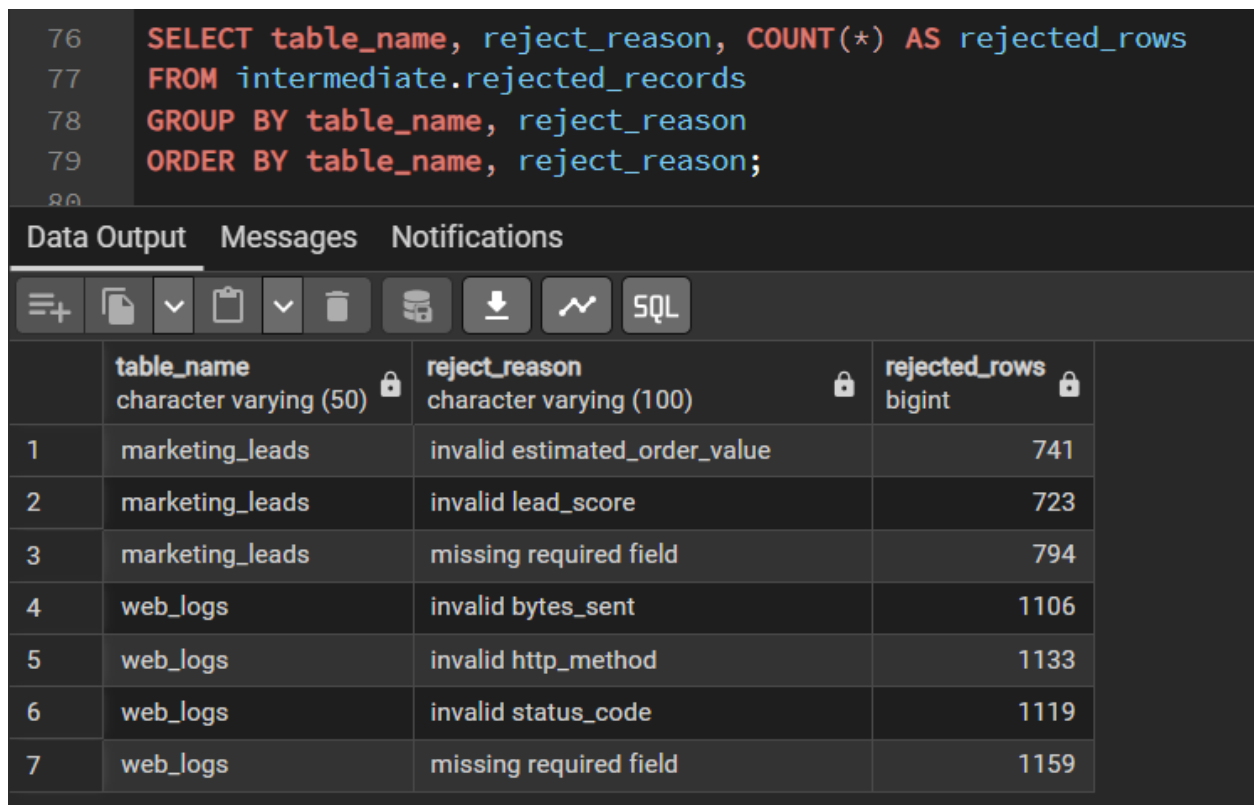


Figure 5: Rejected rows by table and reason

9.5 Logging, Monitoring & Alerting

Every stage writes to logs/pipeline.log and to the console, and records start, end, status, row counts and watermark in metadata.pipeline_run_log. The pipeline-health KPI views (vw_pipeline_execution_metrics, vw_pipeline_run_history, vw_pipeline_latest_run) show load times, failures and data freshness in Power BI. A Windows toast notification reports success or failure of each run.

9.6 Security & Configuration

Connection details are stored in a local .env file (excluded from Git) and validated at start-up. SQL Server uses Windows authentication. Business parameters (volumes, date window, seed, chunk size, paths) are central in config/config.py.

10. Architecture Decisions

Table 10: Architecture Decisions

ADR	Decision	Alternatives Considered	Rationale	Consequences
ADR-01	ELT inside PostgreSQL	ETL in Python (pandas)	Set-based SQL is faster and easier to test; raw data stays available	All transformation logic lives in SQL scripts
ADR-02	Watermark CDC on updated_at	SQL Server built-in CDC; full reloads	Required by the requirements; simple and works on Express edition	Physical deletes are not detected (the source only soft-deletes)
ADR-03	MERGE for every load	TRUNCATE + reload; INSERT only	Idempotent and incremental	Needs PostgreSQL 15+
ADR-04	Galaxy schema with conformed dimensions	One wide table per KPI; single star	Several business processes share Date, Customer, Company, Product	4 fact tables to maintain
ADR-05	SCD2 only for companies	SCD2 everywhere	Only company attributes need history (requirement)	Other dimensions show current values
ADR-06	Monthly range partitions on date_key	No partitioning; yearly partitions	Required by the requirements; fast date filters	Partitions created by partitioning.sql before each load
ADR-07	Empty staging after each successful run	Keep every version in staging	No data lake here; a growing raw copy only costs space and slows the transforms	Staging holds 48 kB after a run instead of growing (230 MB before)
ADR-08	Quarantine table for invalid CSV rows	Drop invalid rows silently	Bad rows must be visible and countable	2,258 leads and 4,517 web logs saved with a reason

11. Quality Requirements (Non-Functional)

Table 11: Quality Requirements (Non-Functional)

NFR ID	Category	Requirement	How It Is Met
NFR-01	Reliability	A failed run can be repeated without manual clean-up	Watermarks only move on success; MERGE loads; transactions
NFR-02	Performance	Incremental run finishes in minutes	CDC extract, partition pruning, indexes on join keys (about 3 minutes measured)
NFR-03	Auditability	Every run and every loaded row is traceable	pipeline_run_log, _loaded_at, rejected_records with a reason
NFR-04	Data quality	Only valid, deduplicated data reaches the warehouse	Intermediate validation, quarantine table, constraints, 186 SQL tests
NFR-05	Security	No credentials in code	.env file excluded from Git; Windows authentication for SQL Server

NFR ID	Category	Requirement	How It Is Met
NFR-06	Maintainability	Easy to change one table's logic	One SQL script per table and step; central config

12. Risks & Technical Debt

Table 12: Risks & Technical Debt

Risk / Debt	Impact	Mitigation / Plan
Single local machine	No high availability	Acceptable for the project; all DDL and code are in Git, so the platform can be rebuilt from scratch
Synthetic data	Results do not reflect a real business	Generators follow the business rules and seasonal patterns from the requirements
Manual execution	Runs depend on someone starting them	One command runs everything; can be scheduled with Windows Task Scheduler if needed
No raw archive	Staging is emptied after each run	Sources keep all data (reload by resetting watermarks); a data lake would keep raw files

13. Glossary

Table 13: Glossary

Term	Definition
Galaxy schema	Several fact tables sharing dimensions (fact constellation).
Conformed dimension	A dimension used with the same meaning by several facts.
Surrogate key	Integer key generated in the warehouse, used by facts to join dimensions.
Partition	A physical piece of a large table, here one per month.
MERGE	SQL statement that inserts new rows and updates existing rows in one step.